Missing Integer Project Documentation

This project solves the “Missing Integer” problem using two different algorithms in Java: a Non-Recursive algorithm and a Recursive algorithm. The main objective is to find the smallest positive integer that does not exist inside a given integer array. Example: Array = {1, 3, 6, 4, 1, 2} Existing positive integers: 1, 2, 3, 4, 6 Therefore, the smallest missing positive integer is: 5 The project compares: 1. A Non-Recursive approach using Boolean Hashing. 2. A Recursive approach using Bubble Sort and Recursion. The project also explains: • Algorithm logic • Step-by-step execution • Complexity analysis • Performance comparison

Main Class

```
public class Main {

    public static void main(String[] args) {

        int[] A = {1, 3, 6, 4, 1, 2};
int[] B = {1, 2, 3};         int[] C
= {-1, -2};

        System.out.println("Non Recursive: "
+ MissingIntegerNonRecursive.findMissing(A));

        System.out.println("Recursive: "                +
MissingIntegerRecursive.findMissing(A));

        System.out.println("Non Recursive: "
+ MissingIntegerNonRecursive.findMissing(B));

        System.out.println("Recursive: "                +
MissingIntegerRecursive.findMissing(B));

        System.out.println("Non Recursive: "
+ MissingIntegerNonRecursive.findMissing(C));

        System.out.println("Recursive: "                +
MissingIntegerRecursive.findMissing(C));
    }
}
```

First Algorithm — Non-Recursive Approach

Idea: The algorithm creates a Boolean array called “present” to mark which positive numbers exist. After marking all existing numbers, the algorithm searches for the first number that is still false. That number represents the missing integer.

Pseudocode

```
1. Let N = length of array A

2. Create Boolean array present of size N + 1

3. For each element x in A:
      If x > 0 AND x <= N:
present[x] = true

4. For i from 1 to N:
      If present[i] == false:
            return i

5. Return N + 1
```

Implementation (Java)

```
public class MissingIntegerNonRecursive {

    public static int findMissing(int[] A) {

        int N = A.length;

        boolean[] present = new boolean[N + 1];

        for (int i = 0; i < N; i++) {

            if (A[i] > 0 && A[i] <= N) {

                present[A[i]] = true;
}         }

        for (int i = 1; i <= N; i++) {

            if (!present[i]) {

                return i;
```

```
                }
        }

                return N + 1;
        }
        }
```

Step-by-Step Analysis

Example Array: {1, 3, 6, 4, 1, 2} Step 1: Create Boolean array: [F, F, F, F, F, F, F] Complexity: Creating the array takes O(N) because the size depends on N. Step 2: Read number 1 → mark present[1] = true Step 3: Read number 3 → mark present[3] = true Step 4: Read number 6 → mark present[6] = true Step 5: Read number 4 → mark present[4] = true Step 6: Read number 1 again → already true Step 7: Read number 2 → mark present[2] = true Final Boolean array: [F, T, T, T, T, F, T] Complexity: The first loop visits every element once. Therefore: O(N) Step 8: Search from index 1: 1 exists 2 exists 3 exists 4 exists 5 does NOT exist Result: Missing Integer = 5 Complexity: The second loop also visits each element once. Therefore: O(N) Final Time Complexity: O(N) + O(N) The dominant complexity is: O(N)

Second Algorithm — Recursive Approach

Idea: The algorithm first sorts the array using Bubble Sort. After sorting, recursion is used to compare the expected smallest positive integer with the current array element.

Pseudocode

```
1. Sort array A using Bubble Sort

2. Call find(A, 0, 1)

Function find(A, index, expected):

      If index >= length:
return expected

      If A[index] == expected:
return find(index + 1, expected + 1)

      Else if A[index] < expected:
return find(index + 1, expected)

      Else:
return expected
```

Implementation (Java)

```
public class MissingIntegerRecursive {

    public static int findMissing(int[] A) {

        bubbleSort(A);

        return find(A, 0, 1);
}

    private static void bubbleSort(int[] A) {

        int N = A.length;

        for (int i = 0; i < N - 1; i++) {

            for (int j = 0; j < N - i - 1; j++) {

                if (A[j] > A[j + 1]) {
```

```
                    int temp = A[j];
A[j] = A[j + 1];
A[j + 1] = temp;
                }
            }
        }
}

    private static int find(int[] A, int index, int expected) {

        if (index >= A.length)
return expected;

        if (A[index] == expected)             return
find(A, index + 1, expected + 1);

        else if (A[index] < expected)
return find(A, index + 1, expected);

        else
return expected;     }
}
```

Bubble Sort Trace

Original Array: 1 3 6 4 1 2 PASS 1: Compare 1 and 3 → no swap Compare 3 and 6 → no swap Compare 6 and 4 → swap Array becomes: 1 3 4 6 1 2 Compare 6 and 1 → swap Array becomes: 1 3 4 1 6 2 Compare 6 and 2 → swap Array becomes: 1 3 4 1 2 6 PASS 2: Compare 1 and 3 → no swap Compare 3 and 4 → no swap Compare 4 and 1 → swap Array becomes: 1 3 1 4 2 6 Compare 4 and 2 → swap Array becomes: 1 3 1 2 4 6 PASS 3: Compare 1 and 3 → no swap Compare 3 and 1 → swap Array becomes: 1 1 3 2 4 6 Compare 3 and 2 → swap Array becomes: 1 1 2 3 4 6 Final Sorted Array: 1 1 2 3 4 6

Recursive Trace

find(A, 0, 1) A[0] = 1 Expected = 1 Move to: find(A, 1, 2) A[1] = 1 1 < 2 Ignore duplicate and move to: find(A, 2, 2) A[2] = 2 Expected = 2 Move to: find(A, 3, 3) A[3] = 3 Expected = 3 Move to: find(A, 4, 4) A[4] = 4 Expected = 4 Move to: find(A, 5, 5) A[5] = 6 Expected = 5 Since 6 > 5: Return 5 Final Result: Missing Integer = 5

Complexity Analysis

Bubble Sort Complexity: The outer loop runs N times. The inner loop also runs approximately N times. Total operations: N × N Therefore: $O(N^2)$ Recursive Search Complexity: The recursive function visits each element once. Therefore: O(N) Overall Complexity: $O(N^2)$ + O(N) The dominant complexity is: $O(N^2)$

Comparison Between Algorithms

Feature	Non-Recursive	Recursive
Technique	Boolean Hashing	Bubble Sort + Recursion
Time Complexity	O(N)	$O(N^2)$
Performance	Faster	Slower
Simplicity	Medium	Easy to understand
Practical Use	Preferred	Educational

Conclusion

The project demonstrates two different approaches for solving the Missing Integer problem. The Non-Recursive algorithm is more efficient because it works in linear time O(N). It is considered the best practical solution. The Recursive algorithm is slower because it uses Bubble Sort with complexity $O(N^2)$, but it is useful for educational purposes because it demonstrates recursion and sorting clearly. The project also highlights the difference between iterative and recursive problem-solving techniques.

Github Link :- https://github.com/AhmedHossam06/Missing_Integer